React Interview Prep

Topic 10 — useContext: Solving Prop Drilling

BEGINNER → INTERMEDIATE → EXPERT

■ Covers	Prop drilling · createContext · Provider · useContext · Re-render problem · Custom hook pattern
■ Series	React Interview Prep — Session 10

by Akshay Chavhan • React Interview Series

1. The Problem — Prop Drilling

Passing props through components that don't need them, just to reach a deeply nested child.

```
function App() {
const user = { name: 'Akshay', role: 'Developer' };
return <Header user={user} />; // passed down
}
function Header({ user }) {
// ■ Header doesn't need user — just a middleman
return <UserAvatar user={user} />;
}
function UserAvatar({ user }) {
// Finally used 2 levels deep
return <img src={`/avatars/${user.name}.png`} />;
}
```

■ ***Prop drilling becomes painful at 3+ levels with multiple values. useContext eliminates the middlemen — any component reads directly.***

2. The 3 Steps of useContext

Step 1 creatContext() → **Step 2 Provider** → **Step 3 useContext()**

Step 1: createContext() — Create the notice board

```
import { createContext } from 'react';

const UserContext = createContext(null); // null = default value

export default UserContext;

// Convention: name it SomethingContext. Export for other files.
```

Step 2: Provider — Put data on the board

```
import UserContext from './UserContext';

function App() {

const user = { name: 'Akshay', role: 'Developer' };

return (


// no prop needed anymore ■

// can read user directly ■


);

}
```

Step 3: useContext() — Read from the board anywhere

```
import { useContext } from 'react';

import UserContext from './UserContext';


function UserAvatar() {

const user = useContext(UserContext); // reads directly ■

return <img src={avatarUrl} />;

}

// No props needed. No middlemen. Direct access.
```

3. Full Example — Theme Toggle

A real working example: theme read AND updated from a deeply nested component.

```
import { createContext, useContext, useState } from 'react';
const ThemeContext = createContext('light');
export default function App() {
const [theme, setTheme] = useState('light');
return (
<ThemeContext.Provider value={{ theme, setTheme }}>
<Page />
</ThemeContext.Provider>
);
}
function Page() {
return <div><Navbar /><Content /></div>;
// Page doesn't receive OR pass any props ■
}
function Navbar() {
const { theme, setTheme } = useContext(ThemeContext);
return (
<nav style={{ background: theme === 'dark' ? '#333' : '#fff' }}>
<button onClick={() =>
setTheme(t => t === 'light' ? 'dark' : 'light')}>
Toggle Theme
</button>
</nav>
);
}
function Content() {
const { theme } = useContext(ThemeContext);
return (
<main style={{ color: theme === 'dark' ? '#fff' : '#000' }}>
Hello Akshay!
</main>
);
}
```

4. The Re-render Problem — Expert

Every consumer re-renders when the Provider value changes. If value is a new object every render — all consumers re-render unnecessarily.

■ **Wrong**

```
// ■ New object on every render
// count change → new value object
// → ALL context consumers re-render
function App() {
const [theme, setTheme] = useState('light');
const [count, setCount] = useState(0);
return (
<ThemeContext.Provider
value={{ theme, setTheme }}>
// new object every render ■
</ThemeContext.Provider>
);
}
```

■ **Correct**

```
// ■ useMemo stabilizes the value
// count change → same value object
// → consumers do NOT re-render
function App() {
const [theme, setTheme] = useState('light');
const [count, setCount] = useState(0);
const val = useMemo(
() => ({ theme, setTheme }),
[theme] // only new obj when theme changes
);
return (
</ThemeContext.Provider>
);
}
```

■ ***Always wrap Provider value in useMemo when it's an object. Otherwise every unrelated state change triggers all consumers.***

5. Custom Hook Pattern — Production Best Practice

Instead of importing the context + useContext everywhere, wrap it in a custom hook. This is the pattern used in real production apps.

```
const AuthContext = createContext(null);
// Custom hook — hides implementation, adds safety check
export function useAuth() {
const context = useContext(AuthContext);
if (!context) {
throw new Error('useAuth must be inside AuthProvider');
}
return context;
}
// Provider component — clean wrapper
export function AuthProvider({ children }) {
const [user, setUser] = useState(null);
const login = useCallback((u) => setUser(u), []);
const logout = useCallback(() => setUser(null), []);
const value = useMemo(
() => ({ user, login, logout }), [user, login, logout]
);
return (
<AuthContext.Provider value={value}>
{children}
</AuthContext.Provider>
);
}
// In any component — super clean, no context import needed
function Navbar() {
const { user, logout } = useAuth(); // ■
return <button onClick={logout}>{user.name}</button>;
}
```

> ***■ Custom hook pattern = cleaner API + safety check + hidden implementation. This is the pattern used in real codebases — always prefer this over raw useContext.***

6. Context vs Props vs Redux/Zustand

Situation	Best choice
Data needed 1-2 levels deep only	Props — simplest, most explicit
Global UI: theme, language, auth, modals	useContext — built-in, no extra library
High-frequency updates (mouse pos, scroll)	Local state — context re-renders all consumers
Complex state with many update sources	Redux or Zustand
Large teams needing strict patterns	Redux — enforces discipline
Small-medium app needing global state fast	Zustand — minimal, fast to set up

■ ***useContext is NOT a replacement for Redux/Zustand. It solves prop drilling. Redux/Zustand solve complex state management.***

7. Interview Q&A;

Q Beginner **Q1. What is useContext and why does it exist?**

➜ useContext solves prop drilling — passing data through many components that don't need it just to reach a deeply nested one. createContext creates a shared data store, Provider puts data into it, and useContext reads it from any component in the tree without prop chains.

Q Beginner **Q2. What are the 3 steps of useContext?**

➜ First — createContext() to create the context object. Second — wrap components with Context.Provider and set its value prop. Third — call useContext(ContextName) inside any component that needs the data. No prop chains needed.

Q Beginner **Q3. What is prop drilling?**

➜ Prop drilling is when data is passed through multiple intermediate components that don't actually use it — just to reach a deeply nested component that does. It creates tight coupling and makes refactoring painful. useContext eliminates these middlemen.

Q Intermediate **Q4. What is the re-render problem with Context?**

➜ Every time the Provider value changes, ALL components consuming that context re-render — even if the specific data they use didn't change. Fix: wrap the value in useMemo so a new object is only created when relevant data actually changes, not on every parent re-render.

Q Intermediate **Q5. What is the custom hook pattern with Context?**

➜ Instead of importing the context and useContext in every file, create a custom hook like useAuth() that wraps useContext internally. This hides implementation, lets you add a safety check for missing Provider, and gives consumers a clean API — just call useAuth() anywhere.

Q Intermediate **Q6. When should you NOT use Context?**

➜ Avoid Context for high-frequency values like mouse position or scroll that change constantly — every consumer re-renders on every change causing performance issues. Also avoid it when state is complex with many interdependencies — use Redux or Zustand instead.

Q Expert **Q7. What is the difference between Context and Redux/Zustand?**

➜ Context is built-in and solves prop drilling for relatively stable values. It has no middleware, no DevTools, no time-travel. Redux enforces strict unidirectional flow with full DevTools and middleware support — great for large teams. Zustand is minimal with direct state updates — great for medium apps. Context alone is not a state manager.

Q Expert **Q8. Can you have multiple contexts in one app?**

➜ Yes — you can nest as many Providers as needed. Common pattern: AuthContext for user, ThemeContext for dark/light, CartContext for e-commerce. Each component reads only the context it needs. Splitting contexts also prevents unnecessary re-renders — a theme change won't re-render auth consumers.